

SAPIENCE · SECURITY REVIEW

Sapience Foil Vault Prediction Market.

A security review of Foil Vault Prediction Market.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Sapience	Foil Vault Prediction Market	Guardian	September 29 – October 24, 2025	October 31, 2025

§ 00

Contents.

82 pages — overview, confidence ranking, scope, 61 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 49 findings	P. 11
	Remediation Review · 12 findings	P. 68
§ 05	Disclaimer	P. 81
§ 06	About Guardian	P. 82

§ 01

Overview.

Guardian reviewed Foil Vault Prediction Market across 2 review rounds. The final report includes 2 Critical, 3 High, 7 Medium, 26 Low, 23 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	sapiencexyz/sapience
CLIENT	METHODOLOGY
Sapience	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Foil Vault Prediction Market	3cbcc2c7c444
PREPARED BY	REMEDIATION REVIEW COMMIT
Guardian	ff05bfb4137f
AUDIT WINDOW	
September 29 – October 24, 2025	
FINAL REPORT	
October 31, 2025	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	2	0	0	0	2
● High	3	0	0	0	3
● Medium	7	0	4	0	3
● Low	26	0	7	0	19
● Informational	23	0	9	0	14
All severities	61	0	20	0	41

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Foil Vault Prediction Market in `sapiencexyz/sapience`.
The scope includes 1,078 source lines across 4 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
packages/protocol/src/vault/PassiveLiquidityVault.sol	415	715
packages/protocol/src/predictionMarket/PredictionMarket.sol	359	504
packages/protocol/src/predictionMarket/Utils/SignatureProcessor.sol	28	39
packages/protocol/src/predictionMarket/resolvers/PredictionMarketUmaResolver.sol	276	358

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	PredictionMarket Signatures Can Be Replayed	Signatures	CRITICAL	RESOLVED	p. 11
C-02	The Vault Available Assets Can Be Inflated	Logical Error	CRITICAL	RESOLVED	p. 12
H-01	Missing ERC721 Overwrite	Compatibility	HIGH	RESOLVED	p. 15
H-02	Unconfirmed Assets Should Not Be Held	Math	HIGH	RESOLVED	p. 16
H-03	Liquidity Vault Is Unable To Receive NFTs	DoS	HIGH	RESOLVED	p. 19
M-01	lastUserInteractionTimestamp DoS	DoS	MEDIUM	RESOLVED	p. 20
M-02	Consolidation Should Not Be Permissionless	Access Control	MEDIUM	RESOLVED	p. 21
M-03	Share Price Calculation Can Be Broken	Logical Error	MEDIUM	ACKNOWLEDGED	p. 22
M-04	Winning Taker Can Be Stuck	Unexpected Behavior	MEDIUM	RESOLVED	p. 23
M-05	Slippage Check Is Insufficient	Logical Error	MEDIUM	ACKNOWLEDGED	p. 25
M-06	Already Settled Market Can Be Matched	Logical Error	MEDIUM	ACKNOWLEDGED	p. 27
L-01	Anyone Can Cause Emission Of AssertionDisputed	Access Control	LOW	RESOLVED	p. 29
L-02	There's No way To Deactivate A Prediction Market	Configuration	LOW	RESOLVED	p. 30

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-03	ERC4626 Standard Broken	Best Practices	LOW	RESOLVED	p. 31
L-04	requestWithdrawal Allowed During Emergency	Validation	LOW	RESOLVED	p. 32
L-05	Missing whenNotPaused Modifiers	Best Practices	LOW	RESOLVED	p. 33
L-06	notProcessingRequests Is Redundant	Gas Optimization	LOW	RESOLVED	p. 34
L-07	onlyManager Modifier Not Used	Gas Optimization	LOW	RESOLVED	p. 35
L-08	Missing orderDeadline Validation	Validation	LOW	RESOLVED	p. 36
L-09	Missing Overrides	DoS	LOW	RESOLVED	p. 37
L-10	PassiveLiquidityVault Deposits Can Be Prevented	DoS	LOW	ACKNOWLEDGED	p. 38
L-11	Critical State Change Without Event	Best Practices	LOW	RESOLVED	p. 39
L-12	approvedAsserters Is Immutable	Best Practices	LOW	ACKNOWLEDGED	p. 40
L-13	Bond Currency In UmaResolver	Configuration	LOW	ACKNOWLEDGED	p. 41
L-14	Centralization Risk	Warning	LOW	ACKNOWLEDGED	p. 42
L-15	Funds Are Frozen If Prediction Can't Be Settled	Logical Error	LOW	ACKNOWLEDGED	p. 43
L-16	Wrong Default Expiration Time	Configuration	LOW	RESOLVED	p. 44
L-17	MIN_DEPOSIT Is Not Flexible	Configuration	LOW	RESOLVED	p. 45
L-18	requestWithdrawal Can Be Partially DOSed	DoS	LOW	RESOLVED	p. 46
L-19	Shares Requested For Withdrawal	Unexpected Behavior	LOW	RESOLVED	p. 47
L-20	MIN_DEPOSIT Can Be Bypassed	Validation	LOW	ACKNOWLEDGED	p. 48

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-21	approveFundsUsage() Can Use Up All Vault Funds	Trust Assumptions	LOW	RESOLVED	p. 49
L-22	Modifying Vault Time Variables Affect Requests	Configuration	LOW	ACKNOWLEDGED	p. 51
L-23	UMA Settlement Is Not Marked As Settled	Error	LOW	RESOLVED	p. 52
I-01	Missing Min Collateral Check For Taker	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 53
I-02	Yield Is Missed	Rewards	INFORMATIONAL	ACKNOWLEDGED	p. 54
I-03	Changing Managers Impacts Signatures	Signatures	INFORMATIONAL	ACKNOWLEDGED	p. 55
I-04	Unused Code	Superfluous Code	INFORMATIONAL	RESOLVED	p. 56
I-05	PassiveLiquidityVault Prediction Exposure Risk	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 57
I-06	The Vault Doesn't Support Any Token	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 58
I-07	MIN_DEPOSIT Is Inaccurate Name	Best Practices	INFORMATIONAL	RESOLVED	p. 59
I-08	Utilization Rate Calculation Can Be In WAD	Rounding	INFORMATIONAL	RESOLVED	p. 60
I-09	Unnecessary makerWon Condition	Best Practices	INFORMATIONAL	RESOLVED	p. 61
I-10	Repeated Storage Reads In PassiveLiquidityVault	Gas Optimization	INFORMATIONAL	RESOLVED	p. 62
I-11	Redundant Assignments	Gas Optimization	INFORMATIONAL	RESOLVED	p. 63
I-12	Unused Custom Error	Best Practices	INFORMATIONAL	RESOLVED	p. 64
I-13	Unused Ownable Import	Best Practices	INFORMATIONAL	RESOLVED	p. 65
I-14	Arbitrary Resolvers Allowed	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 66
I-15	An Already Settled Market Can Be Used	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 67

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDATION REVIEW					
M-01	Missing Manager Compensation	Logical Error	MEDIUM	ACKNOWLEDGED	p. 68
L-01	Wrong Util Ratio Check	Validation	LOW	RESOLVED	p. 69
L-02	Approval Not Adjusted On Withdraws	Validation	LOW	RESOLVED	p. 70
L-03	Potential Risks Because Of <code>_verifyTransfer()</code>	Unexpected Behavior	LOW	RESOLVED	p. 71
I-01	Unused Code	Superfluous Code	INFORMATIONAL	RESOLVED	p. 73
I-02	CEI Not Followed In <code>PredictionMarket.mint()</code>	Best Practices	INFORMATIONAL	RESOLVED	p. 74
I-03	Already Settled Market Can Be Matched	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 75
I-04	IERC721Receiver Is Not Considered Supported	Informational	INFORMATIONAL	RESOLVED	p. 76
I-05	Inaccurate Comment	Best Practices	INFORMATIONAL	RESOLVED	p. 77
I-06	Ineffective Optimization	Gas Optimization	INFORMATIONAL	RESOLVED	p. 78
I-07	User Balance May Drop Below Their Locked Shares	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 79
I-08	Ambiguous Event Emission	Events	INFORMATIONAL	RESOLVED	p. 80

CRITICAL **C-01**

RESOLVED

PredictionMarket Signatures Can Be Replayed

CATEGORY Signatures
LOCATION PredictionMarket.sol
REVIEW Main Review

DESCRIPTION

`PredictionMarket.mint()` is called by the maker and accepts a `takerSignature` to verify the specified taker has approved the prediction. The signed message doesn't include any type of nonce. This allows the same signature to be used as many times as possible until either:

- the allowance of the taker runs out
- the collateral balance of the taker runs out
- the signature expires due to its deadline

A malicious maker can drain the taker balance since most likely the taker will have approved a large amount of tokens to the `PredictionMarket` contract.

This poses a danger for the vault. If it signed a message for a prediction that's to be soon resolved and the signature still hasn't expired, the maker can drain all of its funds by repeatedly calling `mint()`.

RECOMMENDATION

Add a replay protection mechanism, for example nonce.

PROOF OF CONCEPT <https://github.com/GuardianOrg/sapience-team1-1758819539309/pull/1/files>

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

CRITICAL C-02

RESOLVED

The Vault Available Assets Can Be Inflated

CATEGORY	Logical Error
LOCATION	GLOBAL
REVIEW	Main Review

DESCRIPTION

Users can artificially inflate the value of their shares by transferring losing predictions into the system.

For example:

- Eve enters the `PassiveLiquidityVault`
- Eve mints a prediction with the following stats to herself (takes on both the maker and taker side):
- the prediction will settle in the next days
- the prediction will extremely unlikely resolve to yes (something like an alien invasion will start tomorrow)
- the yes side holds a huge amount of liquidity and the no side almost nothing
- Eve transfers the yes side to the `PassiveLiquidityVault` and therefore increases `userCollateralDeposits` and therefore it's share value drastically
- Eve exits the vault and gains from the increased share price
- Soon after that the prediction resolves to No and Eve gets her funds back
- Repeat

Another impact of this issue is causing high utilization rates. For instance:

- the vault holds 2000 assets and 200 of them are deployed (10% utilization)
- the attacker inflates the collateral of the vault with `50_000` assets => `utilizationRate = 50 200 / 52 200 = >96%`
- the rest of the funds of the vault cannot be used anymore.

RECOMMENDATION

If there is going to be only 1 vault ever, you can hardcode its address in the `PredictionMarket` and forbid transfers to it. Otherwise:

1. Introduce the following mappings to the `PredictionMarket` contract

C-02 The Vault Available Assets Can Be Inflated

CONTINUED

```
mapping(uint256 => address) private initialNftMaker;
mapping(uint256 => address) private initialNftTaker;
mapping(address => uint256) private vaultProvidedCollateral;
```

1. Record the initial owners of the NFTs in `_createPrediction()` and also increase the `vaultProvidedCollateral` for the maker and the taker

```
initialNftMaker[makerNftTokenId] = maker;
initialNftTaker[takerNftTokenId] = taker;
vaultProvidedCollateral[maker] += makerCollateral;
vaultProvidedCollateral[taker] += takerCollateral;
```

1. In the `burn()` function check if the owner of the NFT is the initial owner and if yes, decrease the `vaultProvidedCollateral`

```
if (ownerOf(prediction.makerNftTokenId) == initialNftMaker[prediction.makerNftTokenId]) {
    vaultProvidedCollateral[initialNftMaker[prediction.makerNftTokenId]] -=
    prediction.makerCollateral;
}
if (ownerOf(prediction.takerNftTokenId) == initialNftTaker[prediction.takerNftTokenId]) {
    vaultProvidedCollateral[initialNftTaker[prediction.takerNftTokenId]] -=
    prediction.takerCollateral;
}
```

1. Add a public getter function `getVaultProvidedCollateral` to `PredictionMarket.sol` and `IPredictionMarket.sol`

```
function getVaultProvidedCollateral(
    address user
) external view returns (uint256) {
    return vaultProvidedCollateral[user];
}
```

1. Change the `PassiveLiquidityVault._deployedLiquidity()` to use the newly created view function

```
function _deployedLiquidity() internal view returns (uint256) {
    // get vault's owned NFTs and sum the collateral of each for each NFT
    uint256 totalDeployedAmount = 0;
    address[] memory protocols = activeProtocols.values();
    for (
```

C-02 The Vault Available Assets Can Be Inflated

CONTINUED

```
uint256 protocolIndex = 0;
protocolIndex < protocols.length;
protocolIndex++
) {
    address protocol = protocols[protocolIndex];
    IPredictionMarket pm = IPredictionMarket(protocol);
    uint256 userCollateralDeposits = pm.getVaultProvidedCollateral(
    address(this)
    );
    totalDeployedAmount += userCollateralDeposits;
}
return totalDeployedAmount;
}
```

By doing this even if someone transfers tokens to the vault, they will not be counted as deployed liquidity.

Implemented in the fuzzing suite - <https://github.com/GuardianOrg/sapience-fuzz-1758819592413/commit/739e83cd2048ee2b5901c29924a9191981482c3a>

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

HIGH **H-01**

RESOLVED

Missing ERC721 Overwrite

CATEGORY	Compatibility
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/predictionMarket/PredictionMarket.sol
REVIEW	Main Review

DESCRIPTION

The `PredictionMarket` inherits the ERC721 standard as every maker and taker position is an NFT.

However the advantage of using NFTs as positions (the transferability) is not used. It is possible to transfer the NFTs to other accounts however this account will not become the new maker or taker on transfer. Therefore it is not possible to transfer or sell positions.

RECOMMENDATION

Consider to implement logic that allows to transfer positions.

RESOLUTION

Status: Resolved.

<https://github.com/GuardianOrg/sapience-team1-1758819539309/compare/58ef1791a98e4a39955c4be15b60b5b05cfd3c3d...3cbcc2c7c4444d59de19f3c28819e77e45ed7d51>

HIGH **H-02**

RESOLVED

Unconfirmed Assets Should Not Be Held

CATEGORY Math
LOCATION PassiveLiquidityVault.sol
REVIEW Main Review

DESCRIPTION

When users requests deposits, their assets are transferred to the vault contract and the `unconfirmedAssets` variable is increased with the transferred amount. Several different issues happen because of that:

1. The `availableAssets` and `_getAvailableAssets` functions do not subtract the `unconfirmedAssets` from the balance of the contract.

This leads to issues in multiple parts of the codebase:

- It is possible that the manager uses unconfirmed assets in predictions as the `approveFundsUsage` function includes the `unconfirmedAssets` in the available one
- The calculated utilization ratio is wrong if there are any `unconfirmedAssets` in the contract

1. There is a potential exploit when the `emergencyMode` is active:

`PassiveLiquidityVault.emergencyMode()` can be executed when `emergencyMode == true`.

This allows the users to redeem their shares atomically, without any requests. The used price ratio is the following

```
uint256 vaultBalance = _getAvailableAssets();  
if (vaultBalance == 0) revert InsufficientAvailableAssets(shares, 0);  
uint256 withdrawAmount = Math.mulDiv(  
    shares,  
    vaultBalance,  
    totalShares,  
    Math.Rounding.Floor  
);
```

The `_getAvailableAssets()` returns the raw balance of the contract, including `unconfirmedAssets`. This is problematic since the price ratio can be manipulated by depositing a large amount via `requestDeposit()` (which won't be settled by the manager) and later calling `cancelDeposit()` to get back the funds.

H-02 Unconfirmed Assets Should Not Be Held

CONTINUED

The formula for the additional assets needed to drain the whole vault is

$$\Delta \text{assets} = \text{totalAssets} * (\text{totalShares} / \text{attackerShares} - 1)$$

However, the attacker will still need to wait for `expirationTime` to pass, unless their large `requestDeposit()` was executed in the past.

Furthermore, even if the function is used normally, the depositors of the `unconfirmedAssets` don't own shares, which means their assets are distributed to the rest of the share holders.

1. Because the `NAV` of the vault depends on its balance + the deployed assets, some withdrawals may have the correct price ratio, but not enough assets in the vault backing them. Right now, `unconfirmedAssets` are sitting in the contract and can be consumed by these withdrawals, leaving users unable to cancel their requests.

For example:

- There are 5000 assets in the vault and 5000 shares (1:1)
- 80% of the funds (4000) are deployed to protocols
- The contract is left with `balance = 1000` and `deployedLiquidity = 4000`.
- Alice requests a deposit of `1000` assets $\Rightarrow$ `unconfirmedAssets = 1000` and `asset.balanceOf(vault) = 2000`
- Now Bob can withdraw 2000 of their shares for 2000 assets and leave the vault with 0 assets
- If Alice request expires, she won't be able to cancel it because there are not enough assets in the contract to refund her.
- Now Alice cannot perform any operations in the vault because of its one request per time design. She has to wait until new assets enter the vault or supply them herself, leading to potential losses.

— RECOMMENDATION

Instead of storing the unconfirmed assets in the vault, introduce a new contract,

`UnconfirmedAssetsHolder` and transfer the assets to it. You will allow the vault contract to spend the tokens on behalf of `UnconfirmedAssetsHolder` and the vault will:

- transfer the funds back to the user when `cancelDeposit` is executed
- transfer the funds to itself when `processDeposit` is executed

PROOF OF CONCEPT <https://gist.github.com/ZdravkoHr/0544d0ff4c77daaa57c683b86db48bf0>

H-02 Unconfirmed Assets Should Not Be HeldCONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

HIGH **H-03**

RESOLVED

Liquidity Vault Is Unable To Receive NFTs

CATEGORY	DoS
LOCATION	PassiveLiquidityVault.sol
REVIEW	Main Review

DESCRIPTION

To deploy liquidity to the prediction markets, the manager of the vault will periodically call `approveFundsUsage()` and will sign messages which will turn the `PassiveLiquidityVault` into a taker when `PredictionMarket.mint()` is called.

The `PredictionMarket` uses `_safeMint()` to mint NFTs to both the maker and the taker. However, the taker (`PassiveLiquidityVault`) doesn't implement the `onERC721Received()` function. Because of that the transaction reverts, the vault is unable to receive the NFT and its liquidity can never be deployed.

RECOMMENDATION

Implement the `onERC721Received()` in the `PassiveLiquidityVault()`.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

MEDIUM **M-01**

RESOLVED

lastUserInteractionTimestamp DoS

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/b0eb65a2a9b250b4e8aa989f8eda5dc004d120b2/packages/protocol/src/vault/PassiveLiquidityVault.sol#L326-L383
REVIEW	Main Review

DESCRIPTION

As stated out in docs, the `lastUserInteractionTimestamp` is used to add a delay between user request to prevent rapid-fire interactions.

However if the user's request was not processed and canceled instead after 10 minutes (for example because of price changes), this timestamp is not reset. Therefore if a user is unlucky and prices fluctuate a lot as many predictions are settled it may take the user days just to enter the vault.

RECOMMENDATION

Consider to reset the `lastUserInteractionTimestamp` on cancel.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

MEDIUM **M-02**

RESOLVED

Consolidation Should Not Be Permissionless

CATEGORY Access Control

LOCATION PredictionMarket.sol#L241-281 <https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/3cbcc2c7c4444d59de19f3c28819e77e45ed7d51/packages/protocol/src/predictionMarket/PredictionMarket.sol#L241-L281>

REVIEW Main Review

DESCRIPTION

`PredictionMarket consolidatePrediction()` allows the owner of both the maker and taker NFT for a given prediction to receive all of the collateral in exchange of both the tokens, successfully exiting the prediction market.

However, there is no access control applied to the function, so anyone is allowed to call it. Users on the market who hold both maker and taker tokens will always be in danger of being consolidated. They will still receive their collateral, but won't be able to sell the tokens anymore. Moreover, if the NFTs are used in a market contract for users to buy them, an attacker can just call `consolidatePrediction()` to burn the NFTs, leading to unexpected behavior.

RECOMMENDATION

Allow only the owner of the tokens to call the function.

```
if (prediction.maker != prediction.taker)
  revert MakerAndTakerAreDifferent();
+ require (msg.sender == prediction.maker, <<SomeError>>);
```

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

MEDIUM **M-03**

ACKNOWLEDGED

Share Price Calculation Can Be Broken

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/3cbcc2c7c4444d59de19f3c28819e77e45ed7d51/packages/protocol/src/vault/PassiveLiquidityVault.sol#L480-L485
REVIEW	Main Review

DESCRIPTION

The following edge case scenario will break the `PassiveLiquidityVault` :

- X amount of funds are deposited into the vault
- The manager opens positions with $X * UtilRatio$ amount
- Users withdraw $X - X * UtilRatio$ from the vault
- Utilization Ratio is 100% now
- The vault loses all open predictions
- Now the vault owns 0 assets but has a totalSupply of shares > 0
- A new user enters the vault with \$Y assets and gets Y shares in return (1:1 ratio as there are no assets in the vault yet)
- $X > Y$ which means another user who owns $\geq Y$ Shares can steal all funds from the new user

RECOMMENDATION

Make sure to pause the vault forever and redeploy the system if this edge case scenario should ever happen.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

Winning Taker Can Be Stuck

CATEGORY Unexpected Behavior
LOCATION PredictionMarketUmaResolver.sol
REVIEW Main Review

DESCRIPTION

When a prediction is created, the `encodedPredictedOutcomes` are checked by the `PredictionMarketUmaResolver.validatePredictionMarkets()` function. The validation performed there requires the `marketId` for each outcome to be different than 0.

The prediction itself is a parley and the outcomes are treated as its legs, meaning if even one of the outcomes predicted is not correct, the whole prediction should resolve in favor of the taker.

The logic for this is implemented in `PredictionUmaResolver.resolvePrediction()`.

```
if (market.marketId != marketId) {  
    isValid = false;  
    error = Error.INVALID_MARKET;  
    break;  
}  
  
if (!market.settled) {  
    // Do not immediately invalidate; record and continue.  
    hasUnsettledMarkets = true;  
    continue;  
}  
  
bool marketOutcome = market.resolvedToYes;  
if (predictedOutcomes[i].prediction != marketOutcome) {  
    makerWon = false;  
    // Decisive loss on a settled market: return valid with no error  
    return (true, Error.NO_ERROR, makerWon);  
}
```

If any of the markets is not settled yet, the function doesn't return yet, but checks for a losing leg and if it finds one, the parlay is resolved with `makerWon = false`.

M-04 Winning Taker Can Be Stuck

CONTINUED

However, if `market.marketId != marketId`, the parlay is treated as invalid. Because `marketId` can be any value besides 0, it's possible to create parlays that contain `marketId` that will never be wrapped.

At the time of creating the prediction, the taker will see all the legs, but since he joins a parlay, he will assume that even if one of the outcomes is wrong, he will win everything, so the taker will be incentivized to join. Later, even when the markets are settled, the bet will always resolve as invalid and the collateral will remain stuck in the `PredictionMarket`. If the manager doesn't check that before signing, it's possible to even trap the `PassiveLiquidityVault`.

— RECOMMENDATION

Apply the same logic for the `market.marketId != marketId` case like you do for `!market.settled` - if any of the other outcomes are already resolved in the opposite direction, resolve with `makerWon = false`, otherwise return invalid.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

MEDIUM **M-05**

ACKNOWLEDGED

Slippage Check Is Insufficient

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/vault/PassiveLiquidityVault.sol
REVIEW	Main Review

DESCRIPTION

Depositing into or withdrawing from the vault works in a 2-step flow:

1. The user makes a request with a given amount of funds to deposit or withdraw together with the shares to receive or burn.
2. The manager bot executes the request if the result of it would not lead to stealing from current users.

For the deposit flow the user should not provide the shares to mint based on the current share price but a bit less instead as slippage could happen when the vault makes profit.

However the vault could also make a loss and in that case the slippage check is insufficient, for example:

- The vault is empty
- Bob enters the vault with \$100 assets and receives 100 shares in return (share price = \$1)
- Alice creates a deposit request with \$100 assets and expects 98 shares in return (2% slippage)
- A big bet is lost and the vault loses 25% of its assets
- Now the vault has 75 assets and 100 shares
- Therefore the share price is $1 * 100 / 75 = \$1.33$ now
- Bob's shares are now worth: $100 * 75 / 100 = \$75$
- Alice request is processed and she receives 98 Shares with a worth of: $98 * 175 / 198 = \$86.61$
- Bob's shares are now worth: $100 * 175 / 198 = \$88.38$
- Therefore Alice accidentally gifted a big amount of her deposit to Bob

RECOMMENDATION

Consider adding additional `sharesToMint` parameter to `processDeposit` which is provided by the manager and uses the actual price and pass it to `_mint()`.

Then make sure the value is not lower than what the user specified

M-05 Slippage Check Is Insufficient

CONTINUED

```
require(sharesToMint >= request.shares)
```

Apply the same fix for `processWithdrawal()` as well.

PROOF OF CONCEPT <https://github.com/GuardianOrg/sapience-team1-1758819539309/pull/3>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Already Settled Market Can Be Matched

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/b0eb65a2a9b250b4e8aa989f8eda5dc004d120b2/packages/protocol/src/predictionMarket/PredictionMarket.sol#L374
REVIEW	Main Review

— DESCRIPTION

Both `PredictionMarket.fillOrder()` and `PredictionMarket.mint()` allows users to match their prediction against an opposite one.

In the case of `fillOrder()`, the taker matches an order placed by the maker and in the case of `mint()`, the maker matches an order signed by the taker. The orders in both functions have deadlines that make them invalid after expiration, but there is no way to cancel them earlier. If the market outcome becomes known prematurely and before the deadline expires, these orders can be used to create 100% winning predictions and steal money from their creators.

For example, a maker places `NO` order for `Will PEPE hit $1 until the end of 2025` market with deadline of 1 day. However, `PEPE` hits `$1` after 2 hours. Takers are incentivized to fill that order since it's a sure win for them, while makers cannot do anything to stop them.

— RECOMMENDATION

1. Consider to revert in `fillOrder` if a market is already settled.
2. Allow canceling orders at any point in time

```
function cancelOrder(uint256 orderId) external nonReentrant {
    IPredictionStructs.LimitOrderData storage order = unfilledOrders[
        orderId
    ];
    if (order.orderId != orderId) revert OrderNotFound();
    - if (block.timestamp < order.orderDeadline) revert OrderNotExpired();
    if (order.maker != msg.sender) revert MakerIsNotCaller();
```

3. Add a per user `nonce` towards the taker signatures used in `mint()` and expose a function that allows the user to invalidate nonces. Signing contracts, including `PassiveLiquidityVault`, must have a way to call that function as well.

M-06 Already Settled Market Can Be Matched

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/sapience-team1-1758819539309/pull/2/files>

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Anyone Can Cause Emission Of AssertionDisputed

CATEGORY	Access Control
LOCATION	PredictionMarketUmaResolver.sol#L375-380
REVIEW	Main Review

DESCRIPTION

`PredictionMarketUmaResolver.assertionDisputedCallback()` will be called by the UMA oracle when someone disputes an assertion. The resolver doesn't perform any state changes on dispute, it only emits the `AssertionDisputed()` event. However, the function is not properly guarded and anyone can call it, resulting in incorrect event emissions, making offchain listeners think the assertion was disputed when it was not.

RECOMMENDATION

Allow only the oracle to call the function

```
function assertionDisputedCallback(bytes32 assertionId) external {  
+ if (msg.sender != address(config.optimisticOracleV3)) {  
+ revert OnlyOptimisticOracleV3CanCall();  
+ }  
bytes32 marketId = umaSettlements[assertionId].marketId;  
// do nothing on disputes, just emit the event. We wait for the assertion to be resolved  
(truthfully or not) to close the loop.  
emit AssertionDisputed(marketId, assertionId, block.timestamp);  
}
```

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-02**

RESOLVED

There's No way To Deactivate A Prediction Market

CATEGORY	Configuration
LOCATION	PassiveLiquidityVault.sol
REVIEW	Main Review

— DESCRIPTION

`PassiveLiquidityVault.approveFundsUsage()` adds prediction markets to the `activeProtocols` set, but there is no way to remove a protocol from it.

Whenever `_deployedLiquidity()` is called, it will loop through all of the prediction markets, even if they are already not in use, increasing the gas usage and the risk of encountering an OOG exception.

Furthermore, if `pm.getUserCollateralDeposits()` reverts for any of these markets, all flows using `deployedLiquidity()` will experience DOS.

— RECOMMENDATION

Consider adding a way to remove protocols from the set.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-03**

RESOLVED

ERC4626 Standard Broken

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/vault/PassiveLiquidityVault.sol
REVIEW	Main Review

DESCRIPTION

The `PassiveLiquidityVault` inherits the ERC4626 standard but does not follow it's rules.

RECOMMENDATION

Consider to follow the rules of the ERC4626 standard or remove the inheritance.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-04**

RESOLVED

requestWithdrawal Allowed During Emergency

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/b0eb65a2a9b250b4e8aa989f8eda5dc004d120b2/packages/protocol/src/vault/PassiveLiquidityVault.sol#L241-L244
REVIEW	Main Review

DESCRIPTION

The normal `requestWithdrawal` function is still allowed if the `emergencyMode` is active. Depending on the emergency situation this may result in unexpected behavior or may even allow an exploit.

RECOMMENDATION

Consider to add the `notEmergency` modifier to the `requestWithdrawal` function.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-05**

RESOLVED

Missing whenNotPaused Modifiers

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/b0eb65a2a9b250b4e8aa989f8eda5dc004d120b2/packages/protocol/src/vault/PassiveLiquidityVault.sol#L326-L348>

REVIEW Main Review

DESCRIPTION

The `cancelDeposit` and `cancelWithdrawal` functions do not have a `whenNotPaused` modifier and therefore it is not possible to stop them.

This could turn out bad, especially if there is a vulnerability in the `cancelDeposit` as it transfers funds to the user.

RECOMMENDATION

Consider to add `whenNotPaused` modifiers to these functions.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-06**

RESOLVED

notProcessingRequests Is Redundant

CATEGORY	Gas Optimization
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/b0eb65a2a9b250b4e8aa989f8eda5dc004d120b2/packages/protocol/src/vault/PassiveLiquidityVault.sol#L167-L172
REVIEW	Main Review

DESCRIPTION

The `notProcessingRequests` does the same as the `nonReentrant` modifier and is therefore redundant.

RECOMMENDATION

Consider to remove the `notProcessingRequests` modifier

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-07**

RESOLVED

onlyManager Modifier Not Used

CATEGORY	Gas Optimization
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/b0eb65a2a9b250b4e8aa989f8eda5dc004d120b2/packages/protocol/src/vault/PassiveLiquidityVault.sol#L157-L160
REVIEW	Main Review

— DESCRIPTION

The `onlyManager` is not used and instead it's logic is written out in the process functions.

— RECOMMENDATION

Consider to use the `onlyManager` modifier in the process functions.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-08**

RESOLVED

Missing orderDeadline Validation

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/b0eb65a2a9b250b4e8aa989f8eda5dc004d120b2/packages/protocol/src/predictionMarket/PredictionMarket.sol#L374
REVIEW	Main Review

DESCRIPTION

Users can set any value as `orderDeadline`. If a user is not aware that the order can not be canceled before this timestamp passed and just uses the maximum uint all funds are permanently stuck if the order will not be matched.

RECOMMENDATION

Consider to validate the `orderDeadline` parameter against a maximum.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-09**

RESOLVED

Missing Overrides

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/vault/PassiveLiquidityVault.sol
REVIEW	Main Review

DESCRIPTION

The `PassiveLiquidityVault` tries to overwrite and revert in all `ERC4626` functions, but forgot about the convert functions:

- `convertToShares`
- `convertToAssets`

RECOMMENDATION

Overwrite the convert functions as they will DoS anyway because the `totalAssets` function reverts.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-10**

ACKNOWLEDGED

PassiveLiquidityVault Deposits Can Be Prevented

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/vault/PassiveLiquidityVault.sol
REVIEW	Main Review

— DESCRIPTION

The `PassiveLiquidityVault` does not perform normal share price calculations on chain and instead a bot decides if the user's wished assets to shares ratio is allowed or not.

This allows anyone to DoS deposits with a front run, for example:

- The vault is empty
- Eve requests to deposit 100 assets and receive 100 shares in return
- The manager processes eve's transaction
- share price is now $100 / 100 = \$1$ per share
- Bob requests to deposit 100 assets and receive 100 shares in return (so in the current ratio)
- Eve front runs bob's transaction and transfers 1 asset into the contract
- Share price is now $101/100 = \$1.01$ per share
- As bob tries to buy shares in a 1:1 ratio (cheaper than they currently are) the manager will not accept bob's order

The same can happen accidentally when the vault receives profit from a settled prediction.

— RECOMMENDATION

Consider to make users aware of that so they always calculate with a realistic slippage value.

— RESOLUTION

Status: Acknowledged.

LOW **L-11**

RESOLVED

Critical State Change Without Event

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/3cbcc2c7c4444d59de19f3c28819e77e45ed7d51/packages/protocol/src/vault/PassiveLiquidityVault.sol#L634-L636>

REVIEW Main Review

DESCRIPTION

The `toggleEmergencyMode` function performs a critical state change but does not emit an event.

RECOMMENDATION

Consider to emit an event to follow best practices.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-12**

ACKNOWLEDGED

approvedAsserters Is Immutable

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/predictionMarket/resolvers/PredictionMarketUmaResolver.sol>

REVIEW Main Review

DESCRIPTION

There is not function to update the `approvedAsserters` mapping after the deployment of the `PredictionMarketUmaResolver`.

RECOMMENDATION

Consider to add a function to be able to update it if needed.

RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Bond Currency In UmaResolver

CATEGORY	Configuration
LOCATION	PredictionMarketUmaResolver.sol
REVIEW	Main Review

— DESCRIPTION

The `UMA` oracle accepts a bond payment only if the given token is whitelisted.

```
if (cachedCurrencies[currency].isWhitelisted) return true;
```

`PredictionMarketUmaResolver` uses `config.bondCurrency` as bond token.

```
IERC20 bondCurrency = IERC20(config.bondCurrency);
```

The `config` is set only once during deployment. If the `UMA` oracle removes the asset from the whitelist, the resolver contract becomes unusable.

— RECOMMENDATION

Add ownership feature and a function to change the `config`

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

Centralization Risk

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/vault/PassiveLiquidityVault.sol
REVIEW	Main Review

— DESCRIPTION

The `manager` of the `PassiveLiquidityVault` is able to rug the `PassiveLiquidityVault` in multiple ways. For example by creating a request to withdraw all assets of the vault for 1 share and processing it, or by using `approveFundsUsage` to get all funds out of the contract.

This is a centralization risk that could have very bad consequences if the private key behind this EOA falls into the wrong hands.

— RECOMMENDATION

Be aware of this risk and consider to add more checks in the process functions to limit the power of the `manager`.

For example by calculating the estimated share value with on chain conditions and comparing it with the deposited or withdrawn asset amount to make sure they are not completely off.

Or for example by limiting the amount of funds that can withdrawn in a given time span.

Limiting the power of the `manager` to not rug the vault in one transaction gives the owner a bit of time to pause the contract in an emergency situation.

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

Funds Are Frozen If Prediction Can't Be Settled

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/predictionMarket/PredictionMarket.sol
REVIEW	Main Review

— DESCRIPTION

In the case that a prediction can't be settled, for example as it's a bet on which sports team will win but the game is canceled. The prediction can't be burned and the funds of the users are permanently frozen.

— RECOMMENDATION

Consider to add a deadline to predictions at which it can be canceled so that both parties get their funds back.

— RESOLUTION

Status: Acknowledged.

LOW **L-16**

RESOLVED

Wrong Default Expiration Time

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/3cbcc2c7c4444d59de19f3c28819e77e45ed7d51/packages/protocol/src/vault/PassiveLiquidityVault.sol#L144-L145
REVIEW	Main Review

— DESCRIPTION

All comments talk about a default expiration time of 10 minutes, however it is set to 2 minutes instead, which may be too less.

— RECOMMENDATION

Consider to update it to 10 minutes.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

LOW **L-17**

RESOLVED

MIN_DEPOSIT Is Not Flexible

CATEGORY	Configuration
LOCATION	PassiveLiquidityVault.sol#L136
REVIEW	Main Review

DESCRIPTION

The `MIN_DEPOSIT` constant in `PassiveLiquidityVault` is hardcoded to `100e18`. This is not sufficient for supporting wide range of tokens as different tokens have different value and decimals.

RECOMMENDATION

Consider having this variable configurable.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

LOW **L-18**

RESOLVED

requestWithdrawal Can Be Partially DOSed

CATEGORY	DoS
LOCATION	PassiveLiquidityVault.sol#L264-265
REVIEW	Main Review

DESCRIPTION

The `PassiveLiquidityVault.requestWithdrawal()` function requires the withdrawn shares to be more than `MIN_DEPOSIT` unless the users performs a full withdrawal

```
if (shares < balanceOf(msg.sender) && expectedAssets < MIN_DEPOSIT)
    revert AmountTooSmall(expectedAssets, MIN_DEPOSIT);
```

An attacker can leverage this check to DOS honest user withdrawals. For example, if `share:asset = 1` and Alice tries to withdraw 20e18 shares, she should receive 20e18 assets. Now Bob can frontrun her transaction with a 1 wei donation and make the transaction fail because Alice now has `20e18 + 1` shares.

RECOMMENDATION

Consider adding a function which always performs a full withdraw request and also accepts `maxShares` parameter to ensure the redeemed shares are not exceeding a given amount specified by the user.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

LOW **L-19**

RESOLVED

Shares Requested For Withdrawal

CATEGORY	Unexpected Behavior
LOCATION	PassiveLiquidityVault.sol
REVIEW	Main Review

DESCRIPTION

During `requestWithdrawal()`, the shares being withdrawn are not burned or taken out from the account of the user. This allows the holder to transfer them after the request is created. By transferring the shares to different accounts, users can sort of bypass the `InsufficientBalance()` check and create a lot of pending requests.

When `processWithdrawal()` is invoked, the transaction will revert if the user balance is not sufficient. The users who know the shares can be transferred have an edge over the other participants in case a sudden price spike happens from the time the request was submitted until it's executed. They can just transfer the shares out of their account and leave the rest of the users bear a larger loss.

Furthermore, if the manager batches several calls to `processWithdrawal()` together such reverts can temporarily DOS the execution of other withdrawals.

RECOMMENDATION

Consider if shares must be locked when `requestWithdrawal()` is executed.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

LOW **L-20**

ACKNOWLEDGED

MIN_DEPOSIT Can Be Bypassed

CATEGORY	Validation
LOCATION	PassiveLiquidityVault.sol#L264-265
REVIEW	Main Review

— DESCRIPTION

The `MIN_DEPOSIT` check during withdrawals can be bypassed by using different accounts. Users can transfer 1 wei of shares to many different accounts and request withdrawal from them. Because 1 wei is their whole balance, the check will be successfully bypassed and they will be able to spam the withdrawal queue.

— RECOMMENDATION

Reconsider if the current check is good enough for your goals.

— RESOLUTION

Status: Acknowledged.

LOW **L-21**

RESOLVED

approveFundsUsage() Can Use Up All Vault Funds

CATEGORY	Trust Assumptions
LOCATION	PassiveLiquidityVault.sol
REVIEW	Main Review

DESCRIPTION

In `approveFundsUsage()` there is validation performed to ensure the vault utilization after funds are approved doesn't surpass the `maxUtilizationRate`

```
uint256 newUtilization = ((deployedLiquidity + amount) * BASIS_POINTS) /  
totalAssetsValue;  
if (newUtilization > maxUtilizationRate)  
revert ExceedsMaxUtilization(newUtilization, maxUtilizationRate);
```

However, the real utilization increases only after funds are actually pulled from the contract. If a second approval to a different protocol is executed before the previous one pulled the funds, it's possible to use up all the funds. For example:

- Vault has 1000 assets
- `approveForFunds()` gives 800 approval to protocol A (80%)
- `approveForFunds()` gives 200 approval to protocol B (20%)

If there are no funds pulled in between the two calls, both of them will succeed and the utilization will end up being 100%, leaving the vault with no assets.

Furthermore, the arbitrary `protocol` parameter gives a lot of power to the manager as they can technically provide a malicious contract and drain the whole vault, which can be catastrophic if the manager account becomes compromised.

RECOMMENDATION

1. Include the `allowance` of each protocol towards the `deployedLiquidity` value as they are funds that can be pulled at any time.
2. Consider having either immutable protocols or at least make sure the caller of `approveFundsUsage()` is a proper multisig.

L-21 approveFundsUsage() Can Use Up All Vault Funds

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

LOW **L-22**

ACKNOWLEDGED

Modifying Vault Time Variables Affect Requests

CATEGORY Configuration
LOCATION PassiveLiquidityVault.sol
REVIEW Main Review

DESCRIPTION

When `setInteractionDelay()` and `setExpirationTime()` functions of the `PassiveLiquidityVault` are executed, the storage variables `interactionDelay` and `expirationTime` are being changes.

When a request is created, the `block.timestamp` is saved in `pendingRequests[user].timestamp` and `lastUserInteractionTimestamp[user]`. On further interactions, `expirationTime` and `interactionDelay` are added towards these timestamp to ensure the needed delay has passed.

Modifying one of the two variables will affect all pending requests.

RECOMMENDATION

Consider adding the `expirationTime` and `interactionDelay` when creating the request or performing an interaction and then only check if the timestamp has passed.

RESOLUTION

Status: Acknowledged.

LOW **L-23**

RESOLVED

UMA Settlement Is Not Marked As Settled

CATEGORY	Error
LOCATION	PredictionMarketUmaResolver.sol#L353-355
REVIEW	Main Review

DESCRIPTION

When UMA resolves an assertion, the `PredictionMarketUmaResolver` updates the `WrappedMarket` as settled but does not set `umaSettlements[assertionId].settled` to true. Although on-chain logic currently does not use this flag, the stored struct can mislead off-chain indexers or future logic that might rely on it, creating inconsistent state.

```
if (assertedTruthfully) {  
    market.settled = true;  
    market.resolvedToYes = umaSettlements[assertionId].resolvedToYes;  
    // Missing: umaSettlements[assertionId].settled = true;  
}
```

RECOMMENDATION

Set `umaSettlements[assertionId].settled = true` in `assertionResolvedCallback()` when `assertedTruthfully` is true.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

INFORMATIONAL I-01

ACKNOWLEDGED

Missing Min Collateral Check For Taker

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/b0eb65a2a9b250b4e8aa989f8eda5dc004d120b2/packages/protocol/src/predictionMarket/PredictionMarket.sol#L119-L120>

REVIEW Main Review

DESCRIPTION

The maker collateral amount is checked to be $\geq$ `minCollateral` but the taker collateral is not.

RECOMMENDATION

Consider to check both the maker and taker collateral to be at least the `minCollateral`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-02**

ACKNOWLEDGED

Yield Is Missed

CATEGORY	Rewards
LOCATION	GLOBAL
REVIEW	Main Review

— DESCRIPTION

The system works with USDe as collateral but doesn't stake any of it and therefore misses a lot of yield.

— RECOMMENDATION

Consider to stake USDe to be more capital efficient.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Changing Managers Impacts Signatures

CATEGORY Signatures
LOCATION PassiveVaultLiquidity.sol
REVIEW Main Review

— DESCRIPTION

The manager in `PassiveLiquidityVault` will sign messages to deploy the vault liquidity to prediction markets. Changing the manager via `setManager()` will cause:

- all signed, but not yet executed messages to become invalid
- all previously signed messages by the new manager to become valid

— RECOMMENDATION

Keep that in mind before using `setManager()`

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

Unused Code

CATEGORY	Superfluous Code
LOCATION	GLOBAL
REVIEW	Main Review

DESCRIPTION

There is unused code in multiple parts of the system.

Imports:

- `Ownable` and `ISapienceStructs` in the `PredictionMarket` contract
- `IPredictionStructs` in the `PassiveLiquidityVault` contract

Errors:

- `PassiveLiquidityVault`
- `OnlyOwner`
- `InvalidIndex`
- `ProcessingInProgress`
- `InvalidCaller`
- `PredictionMarketUmaResolver`
- `MarketNotDisputed`
- `MarketNotOpen`
- `InvalidCaller`
- `MarketAlreadyWrapped`
- `MarketNotSettled`

RECOMMENDATION

Consider to use or remove unused code.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

INFORMATIONAL I-05

ACKNOWLEDGED

PassiveLiquidityVault Prediction Exposure Risk

CATEGORY	Warning
LOCATION	GLOBAL
REVIEW	Main Review

— DESCRIPTION

The `PassiveLiquidityVault` acts as something similar to a market maker in the sense that it takes on the opposite side of a bet.

As the quotes for the vault (market price) will always be a bit better than for the user and as the vault has open positions on both sides of many different predictions. It should sum up in the end and the vault should make profit under normal conditions.

However if the vault freely allows users to open bets with any liquidity amount the vault has available than there is a big risk of having too much exposure on a single bet or just a few bets.

For example if the vault owns \$150k and someone creates a big bet that locks up \$100k and the bet is lost than the vault lost a significant share of it's liquidity.

— RECOMMENDATION

Be aware of this risk and make sure that the amount of liquidity the `PassiveLiquidityVault` provides to to a single bet is limited based on the amount of liquidity the vault owns.

Exposure always needs to be distributed enough or otherwise the vault could quickly lose a lot of it's funds with just a bit of bad luck.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

The Vault Doesn't Support Any Token

CATEGORY	Informational
LOCATION	PassiveLiquidityVault.sol
REVIEW	Main Review

— DESCRIPTION

During the kickoff call, it was mentioned that the vault is expected to work with any ERC20 token, but the current implementation doesn't support:

- tokens with `decimals != 18` because of `MIN_DEPOSIT`
- tokens with fee on transfer features

— RECOMMENDATION

Keep that in mind before deploying.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

RESOLVED

MIN_DEPOSIT Is Inacurate Name

CATEGORY	Best Practices
LOCATION	PassiveLiquidityVault.sol#136
REVIEW	Main Review

DESCRIPTION

The `MIN_DEPOSIT` constant in `PassiveLiquidityVault` is a minimum amount of tokens that has to be requested during deposits and withdrawals. Currently the name suggests it will be only used on deposits.

RECOMMENDATION

Consider changing the constant to `MIN_REQUEST_AMOUNT`

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

INFORMATIONAL I-08

RESOLVED

Utilization Rate Calculation Can Be In WAD

CATEGORY Rounding
LOCATION PassiveLiquidityVault.sol
REVIEW Main Review

DESCRIPTION

Currently the utilization rate is represented in `BASIS_POINTS = 1e4`. In result, there is a precision loss of up to `0.01%` happening.

RECOMMENDATION

You can use `WAD` instead of `BASIS_POINTS` for the utilization calculation to reduce the precision loss experienced.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

INFORMATIONAL I-09

RESOLVED

Unnecessary makerWon Condition

CATEGORY	Best Practices
LOCATION	PredictionMarketUmaResolver.sol#L193
REVIEW	Main Review

DESCRIPTION

At the end of `PredictionMarketUmaResolver.resolvePrediction()` it's checked if there are any unsettled markets.

```
if (isValid && hasUnsettledMarkets && makerWon) {  
    // No decisive loss found, but at least one market is unsettled  
    isValid = false;  
    error = Error.MARKET_NOT_SETTLED;  
}
```

The `makerWon` is unnecessary - it's always gonna be true because the only code path that sets it to `false` returns afterwards.

RECOMMENDATION

You can remove `makerWon` from the if condition.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

INFORMATIONAL **I-10**

RESOLVED

Repeated Storage Reads In PassiveLiquidityVault

CATEGORY	Gas Optimization
LOCATION	PassiveLiquidityVault.sol
REVIEW	Main Review

DESCRIPTION

The `PassiveLiquidityVault.requestWithdrawal()` function and `PassiveLiquidityVault.requestDeposit()` read the same storage-backed values multiple times within a single execution path. Specifically, `pendingRequests[msg.sender]` is accessed twice to read fields in both functions and `balanceOf(msg.sender)` is used three times in `requestWithdrawal()`. Each access incurs an SLOAD. Since these values do not change within the function prior to use, caching them in local variables would reduce gas usage.

RECOMMENDATION

Cache storage reads into local variables and reuse them. For example:

- `uint256 bal = balanceOf(msg.sender);` (for `requestWithdrawal()`)
- `PendingRequest storage req = pendingRequests[msg.sender];` (for both functions)

Then use `bal` and `req` consistently throughout the function.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

INFORMATIONAL **I-11**

RESOLVED

Redundant Assignments

CATEGORY Gas Optimization
LOCATION PassiveLiquidityVault.sol#L191-192
REVIEW Main Review

DESCRIPTION

Several state variables are initialized with default values at declaration and then set again in the constructor to identical values. This causes unnecessary SSTOREs and gas usage without changing behavior.

```
uint256 public maxUtilizationRate = 8000; // 80%
uint256 public interactionDelay = 1 days;
...
constructor(...) {
...
manager = _manager;
maxUtilizationRate = DEFAULT_MAX_UTILIZATION_RATE; // same as 8000
interactionDelay = DEFAULT_INTERACTION_DELAY; // same as 1 days
}
```

RECOMMENDATION

Remove the constructor assignments for maxUtilizationRate and interactionDelay if they already equal the DEFAULT_* values.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/779>

INFORMATIONAL **I-12**

RESOLVED

Unused Custom Error

CATEGORY	Best Practices
LOCATION	PassiveLiquidityVault.sol#L62
REVIEW	Main Review

DESCRIPTION

The custom error `OnlyOwner(address caller, address owner)` is declared in `PassiveLiquidityVault`, but never used in any revert paths. Unused custom errors add noise and bloat the bytecode.

RECOMMENDATION

Remove the unused error.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

INFORMATIONAL I-13

RESOLVED

Unused Ownable Import

CATEGORY	Best Practices
LOCATION	PredictionMarket.sol#L9
REVIEW	Main Review

DESCRIPTION

The `PredictionMarket.sol` file imports OpenZeppelin's `Ownable`, but the contract does not inherit from it nor reference any symbols from that file. Unused imports increase code noise and can mislead reviewers into assuming ownership controls exist.

RECOMMENDATION

Remove the unused import.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/768>

INFORMATIONAL **I-14**

ACKNOWLEDGED

Arbitrary Resolvers Allowed

CATEGORY Warning
LOCATION PredictionMarket.sol
REVIEW Main Review

DESCRIPTION

`PredictionMarket._createPrediction()` can be executed with an arbitrary `resolver`. This allows the usage of malicious resolvers which may revert on resolution or control who wins the prediction.

RECOMMENDATION

Make sure the takers, especially the vault, check the resolver before taking an order.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-15

ACKNOWLEDGED

An Already Settled Market Can Be Used

CATEGORY Validation
LOCATION PredictionMarketUmaResolver.sol
REVIEW Main Review

DESCRIPTION

`PredictionMarketUmaResolver.validatePredictionMarkets()` doesn't check if the given market of an outcome has already been settled. This allows the creation of parleys with an already known outcome.

RECOMMENDATION

Consider validating if the market is settled

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Missing Manager Compensation

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/f33e8d94d3761ef0d031b31426984340bdf08a18/packages/protocol/src/vault/PassiveLiquidityVault.sol
REVIEW	Remediation Review

DESCRIPTION

The `manager` has to pay gas to process requests but is not compensated to do so.

RECOMMENDATION

Consider to let users pay a gas fee to the manager.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Wrong Util Ratio Check

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/f33e8d94d3761ef0d031b31426984340bdf08a18/packages/protocol/src/vault/PassiveLiquidityVault.sol#L656
REVIEW	Remediation Review

DESCRIPTION

The `approveFundsUsage` function tries to check if the utilization ratio after this approval exceeds the configured `maxUtilizationRate`.

However the calculation is only based on the total approvals to markets and excludes the already deployed liquidity (`getUserCollateralDeposits`).

RECOMMENDATION

Include the funds from the `getUserCollateralDeposits` calls in this calculation.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

LOW **L-02**

RESOLVED

Approval Not Adjusted On Withdraws

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/main/packages/protocol/src/vault/PassiveLiquidityVault.sol
REVIEW	Remediation Review

DESCRIPTION

Approvals of the vault to the given prediction markets needs to be configured manually with the `approveFundsUsage`. As it is not adjusted automatically when users exit the system it is possible that unconfirmed assets are utilized.

For example:

- eve requests a deposits of 1000 assets
- the deposit is processed
- 80% so 800 assets are approved to the prediction market
- alice requests a deposit of 500 assets
- eve front runs the transaction and requests a withdraw of 999 assets
- alice deposit request goes through
- eve's withdraw request is processed
- eve creates a prediction and 501 assets are pulled from the vault (possible as the approval still exists from when eve had 1000 assets in the vault)
- now all of alice funds are used but she has no shares in the vault which means if her deposit fails all of her funds were stolen / belong to eve now as she owns 1 share in the system

RECOMMENDATION

Consider to automatically decrease the utilization rate after a withdrawal if the available assets < the max utilization rate.

PROOF OF CONCEPT <https://github.com/GuardianOrg/sapience-fuzz-1758819592413/blob/3dd4b55fa3354efc023c35f3e0431a988c8f2dbf/packages/protocol/test/fuzzing/FoundryPlayground.sol#L17-L34>

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

LOW **L-03**

RESOLVED

Potential Risks Because Of `_verifyTransfer()`

CATEGORY	Unexpected Behavior
LOCATION	PredictionMarket.sol#L524
REVIEW	Remediation Review

— DESCRIPTION

When the `PredictionMarket` NFTs are transferred between accounts, the `_update()` function calls `to.supportsInterface()` if the `to` address has code.

```
if (addr.code.length == 0) {
    return false;
}
// Use ERC-165 standard interface detection
try IERC165(addr).supportsInterface(type(IPassiveLiquidityVault).interfaceId) returns (bool supported) {
    return supported;
} catch {
    return false;
}
```

This allows any account to reject transfers, even if they were executed via `transferFrom()` and not `safeTransferFrom()`. There is a check for the size of the code of the account, however since EIP-7702, EOAs can have code associated with them to be executed when they are called. Therefore, any EOA or a contract can reject a transfer.

The `supportsInterface()` also gives the control flow to the `to` address even when `transferFrom()` is used, which may be unexpected, since most NFTs are not doing it.

Furthermore, `_verifyTransfer()` is executed before all changes in `PredictionMarket._update()` are applied. This can lead to loss of funds for users in some very specific edge cases. For example:

- Alice and Bob have 5000 collateral each
- they are respectively a maker and taker for 1000 collateral
- Prediction is settled and taker wins
- Alice transfers the maker NFT to Bob
- Bob uses the hook to call `burn()`
- `burn()` will reduce both of their collaterals with 1000

L-03 Potential Risks Because Of `_verifyTransfer()`

CONTINUED

• the rest of the `_update()` function will reduce Alice's collateral with 1000 more and will add it to Bob
In result Alice lost her collateral twice.

— RECOMMENDATION

Follow the CEI pattern in the `_update()` function and be very explicit in the documentation that `transferFrom()` gives the control flow to the recipient, so third party integrators are prepared for any related risks. As an additional safety measure, you can have the gas forwarded for the `supportsInterface` call capped.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

INFORMATIONAL **I-01**

RESOLVED

Unused Code

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/sapience-team1-1758819539309/blob/f33e8d94d3761ef0d031b31426984340bdf08a18/packages/protocol/src/predictionMarket/resolvers/PredictionMarketUmaResolver.sol
REVIEW	Remediation Review

DESCRIPTION

There are unused errors in the `PredictionMarketUmaResolver`:

- `MarketNotDisputed`
- `MarketNotOpen`
- `InvalidCaller`
- `MarketAlreadyWrapped`
- `MarketNotSettled`

RECOMMENDATION

Consider to remove the unused errors.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

INFORMATIONAL I-02

RESOLVED

CEI Not Followed In PredictionMarket.mint()

CATEGORY	Best Practices
LOCATION	PredictionMarket.sol#L174
REVIEW	Remediation Review

DESCRIPTION

The CEI pattern is not followed in the `PredictionMarket.mint()` function. It first checks the nonce, then calls `isValidSignature()` and finally updates the maker's nonce. While this should be generally safe, because `isValidSignature()` is called on the taker, it increases the risk of signature replays if some taker has non-standard implementation.

RECOMMENDATION

Consider updating the nonce immediately after checking it.

```
- if (mintPredictionRequestData.makerNonce != nonces[mintPredictionRequestData.maker]) {  
+ if (mintPredictionRequestData.makerNonce != nonces[mintPredictionRequestData.maker]++) {  
  revert InvalidMakerNonce();  
}
```

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

INFORMATIONAL I-03

ACKNOWLEDGED

Already Settled Market Can Be Matched

CATEGORY Warning
LOCATION PredictionMarket.sol
REVIEW Remediation Review

DESCRIPTION

As a fix for [M-06](https://www.notion.so/guardianaudits/M-06-27e8bda5828c8109b1dbc9e9ea4bc0af?source=copy_link) orders are now cancellable at any point in time, not only after they expire.

While this protects users from having their orders executed unexpectedly, the `mint()` function is still susceptible to that.

RECOMMENDATION

Acknowledge or add a fix for `mint()` as well.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

IERC721Receiver Is Not Considered Supported

CATEGORY	Informational
LOCATION	PassiveLiquidityVault.sol#L828-833
REVIEW	Remediation Review

DESCRIPTION

When `PassiveLiquidityVault.supportsInterface()` is called for the `IERC721Receiver` interface, the result will be `false` because that interface is not included in the function, even though the vault implements it.

RECOMMENDATION

Consider including the interface in the `supportsInterface()` function.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

INFORMATIONAL **I-05**

RESOLVED

Inaccurate Comment

CATEGORY	Best Practices
LOCATION	PassiveLiquidityVault.sol#L197
REVIEW	Remediation Review

DESCRIPTION

The comment above `PassiveLiquidityVault.decimals()` says that an offset is added to the underlying asset's decimals, but that is not true as the function returns `_underlyingDecimals` unchanged.

```
* @dev Decimals are computed by adding the decimal offset on top of the underlying asset's
decimals. This
* "original" value is cached during construction of the vault contract. If this read operation
fails (e.g., the
* asset has not been created yet), a default of 18 is used to represent the underlying asset's
decimals.
*
* See {IERC20Metadata-decimals}.
*/
function decimals() public view virtual override returns (uint8) {
    return _underlyingDecimals;
}
```

RECOMMENDATION

Consider changing that part of the comment.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

INFORMATIONAL I-06

RESOLVED

Ineffective Optimization

CATEGORY	Gas Optimization
LOCATION	PassiveLiquidityVault.sol#L318-323 PassiveLiquidityVault.sol#L358-363
REVIEW	Remediation Review

DESCRIPTION

The code was updated in a way that when a deposit or withdrawal request is performed in the `PassiveLiquidityVault`, each field of `pendingRequest` is updated separately.

```
request.user = msg.sender;  
request.isDeposit = true;  
request.shares = expectedShares;  
request.assets = assets;  
request.timestamp = block.timestamp;  
request.processed = false;
```

This version of the code uses 5 `SSTORE` compared to using only 1 with the previous one.

RECOMMENDATION

Consider storing everything at once.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

INFORMATIONAL **I-07**

ACKNOWLEDGED

User Balance May Drop Below Their Locked Shares

CATEGORY	Informational
LOCATION	PassiveLiquidityVault.sol
REVIEW	Remediation Review

— DESCRIPTION

The `_update()` function in `PassiveLiquidityVault` was overridden to prevent transfers of locked shares. However, there is a way for the user balance to drop below these locked shares during emergencies. For example:

- User has 1000 shares
- They request a withdrawal of 500 shares
- Emergency mode activates and they withdraw their whole balance
- Now their balance is 0, but the locked shares are still 500

The user would have to clear their withdrawal request manually when the contract is not paused.

— RECOMMENDATION

No code changes needed, but would be good if it's documented.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-08

RESOLVED

Ambiguous Event Emission

CATEGORY	Events
LOCATION	PassiveLiquidityVault.sol
REVIEW	Remediation Review

DESCRIPTION

The `UtilizationRateUpdated` event is emitted when `setMaxUtilizationRate` changes the maximum utilization or when `approveFundsUsage()` is called. These are two different use cases, but the event emitted is the same.

For example:

- Admin sets the utilization to 80%, therefore `UtilizationRateUpdated(0, 80e18)` is emitted
- 600 of 1000 funds are approved, so `UtilizationRateUpdated(0, 60e18)` is emitted

Notice that for the last emission, the second parameter is the projected utilization, not the actual one.

RECOMMENDATION

Consider separating the two use cases to emit different events.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/791>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.